Rising Waters-Flood Prediction System

Project Description:

The Flood Prediction System is a machine learning-based application developed to predict the possibility of floods using historical environmental and rainfall data. The system analyzes important parameters such as rainfall, river water levels, humidity, and other related factors to generate accurate flood predictions. Data preprocessing techniques such as handling missing values, removing duplicate records, and feature scaling are applied to improve model performance. Multiple machine learning algorithms are trained and evaluated, and the best-performing model is used for prediction. This system helps provide early flood warnings, supports disaster management, and assists authorities in making timely decisions to reduce the impact of floods.

.

Scenarios

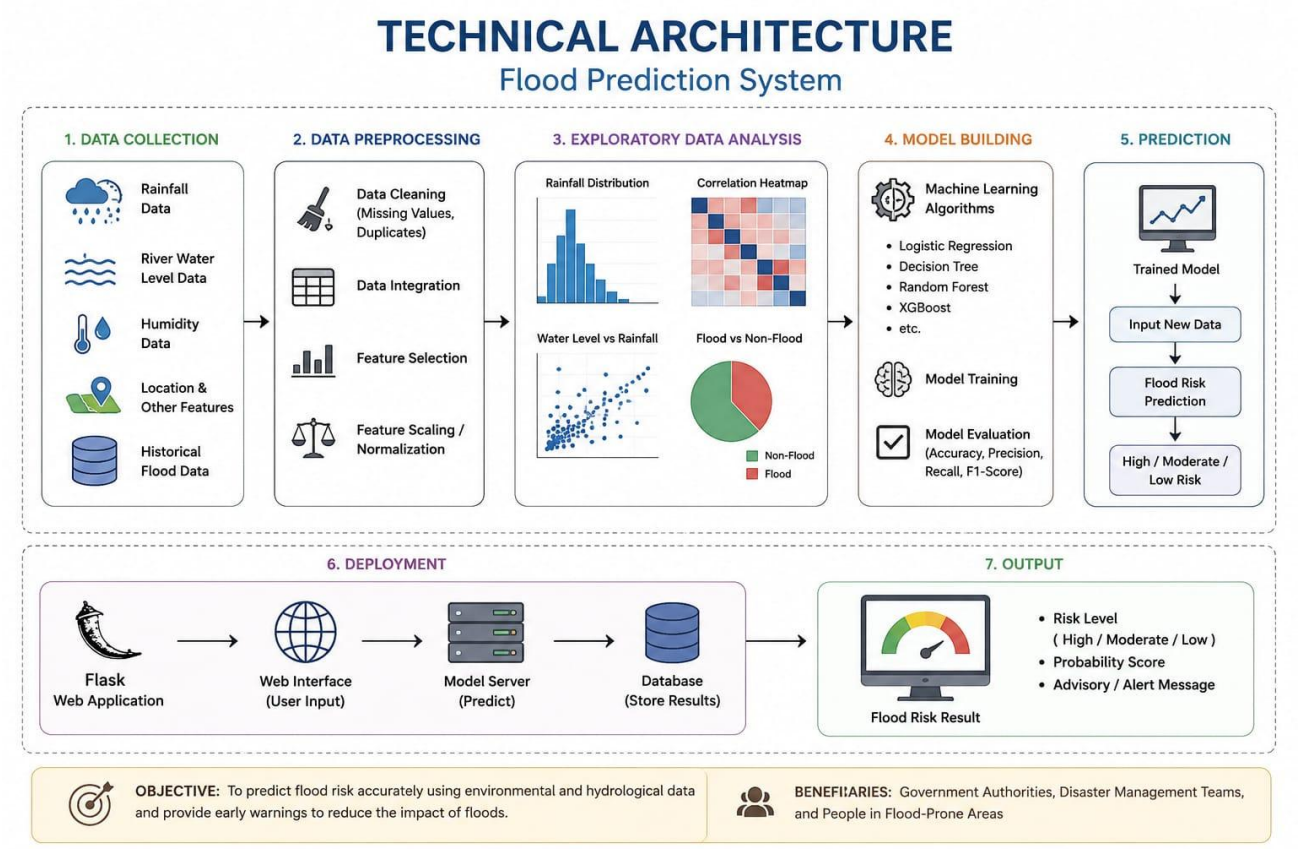
Scenario 1: user enters rainfall, humidity, and river water level data into the system. The machine learning model analyzes the input and predicts whether there is a high or low risk of flooding, providing an early warning to the user.

Scenario : 2Government authorities use the system to monitor flood-prone areas. Based on the predicted flood risk, they can issue alerts, plan evacuations, and arrange emergency resources before flooding occurs.

Scenario 3: Residents in flood-prone regions use the system to check flood risk before traveling or carrying out daily activities. The prediction results help them take preventive measures and stay safe during adverse weather conditions.

.

Technical Architecture:



Description:

The Flood Prediction System follows a structured architecture consisting of data collection, data preprocessing, exploratory data analysis, machine learning model development, prediction, deployment, and result visualization. Historical flood data and environmental parameters such as rainfall, river water level, and humidity are collected and preprocessed by handling missing values, removing duplicates, and scaling features. The processed data is used to train and evaluate multiple machine learning models. The best-performing model predicts flood risk for new input data. The system is deployed through a web interface, allowing users to enter data and receive real-time flood risk predictions along with alert messages to support timely decision-making.

Ee description ni image kinda direct ga paste cheyochu.
(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Groq API Familiarity:** Groq API Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide
- **Ngrok for Public Deployment:** Ngrok Documentation

Project Workflow:

Milestone 1: Data Collection & Preprocessing

Activity 1.1: Generate a Groq API key and configure it securely in the project environment.

- **Activity 1.2:** Collect the flood prediction dataset from reliable sources.
- **Activity 1.3:** . Load the dataset into Google Colab using Pandas.
- **Activity 1.**4 :Handle missing values and remove duplicate records.
-

Milestone 2: Data Analysis & Visualization

- **Activity 2.1:** perform exploratory data analysis(EDA)
- **Activity 2.2:** Generate graphs and visualization to understand data patterns
-

Milestone 3: Model Development

- **Activity 3.1:** Train multiple machine learning models such as Logistic Regression, Decision Tree, Random Forest, KNN, and XGBoost.

Milestone 4: Flood Prediction

- **Activity 4.1:** Accept user input for flood-related parameters.
- **Activity 4.2:** Predict flood risk using the trained machine learning model.
-

Milestone 5: Deployment & Testing

- **Activity 5.1:** Test the system with different input values and verify prediction accuracy.
- **Activity 5.2:** Document the results and prepare the project for deployment.
- **Milestone 6: Conclusion**

The Flood Prediction System was successfully developed and tested using machine learning techniques. The system effectively analyzes historical flood data and environmental factors to predict flood risk with high accuracy. Data preprocessing, model training, and performance evaluation improved the reliability of predictions. This project can support early flood warning systems, help disaster management authorities make timely decisions, and reduce the impact of floods on human life and property. In the future, the system can be enhanced by integrating real-time weather data, IoT sensors, and GIS mapping for more accurate and efficient flood prediction

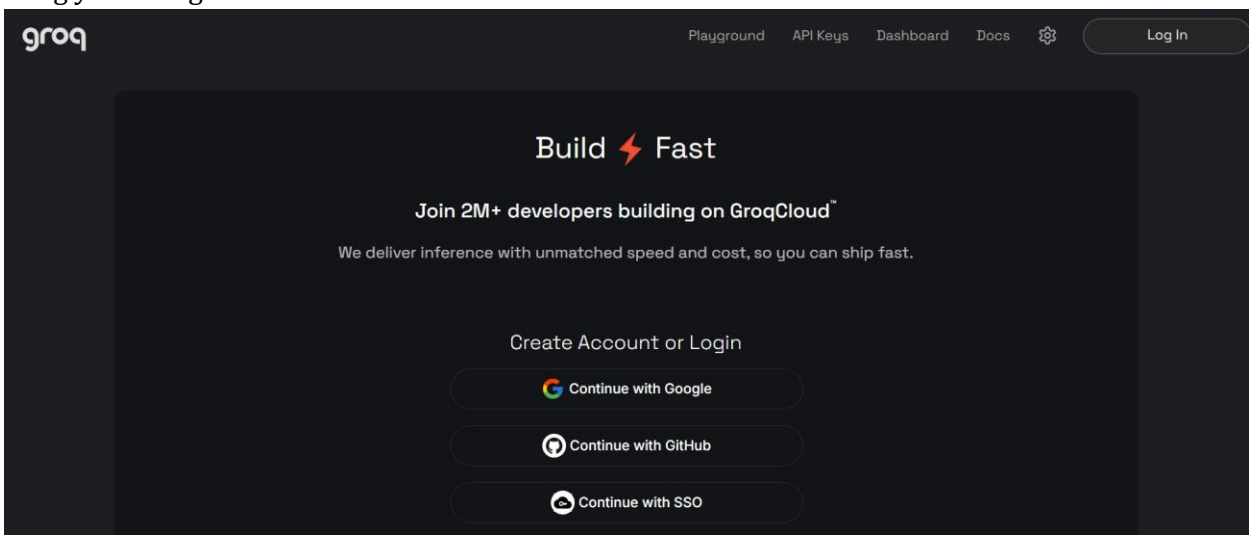
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our social media content generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating captions, hashtags, and CTAs across different platforms and tones.

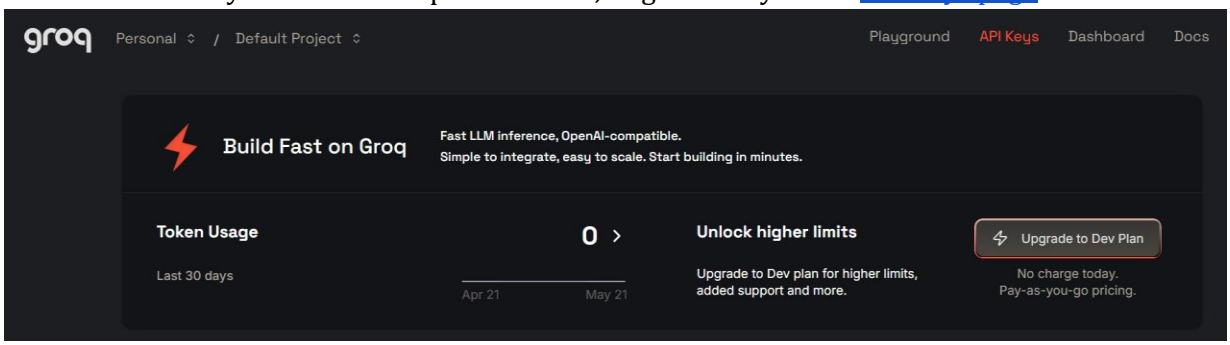
Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

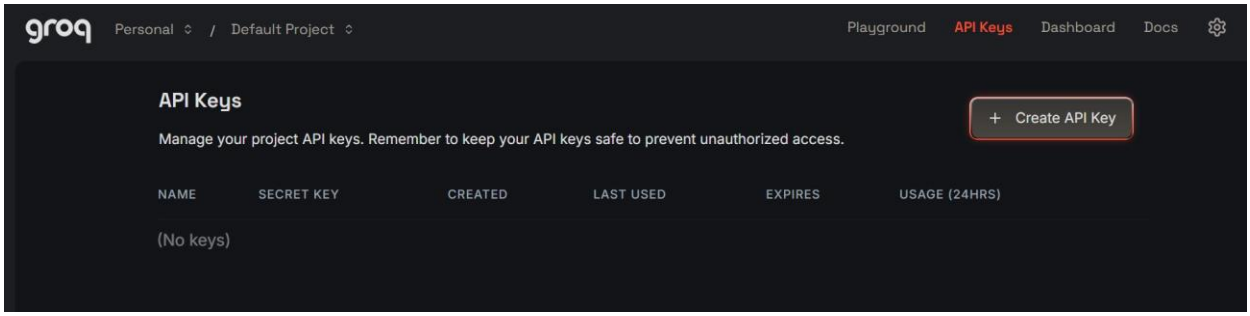
• **Step 1 — Create a Groq Account**: Visit https://console.groq.com and sign up for a free account using your Google account or email address.



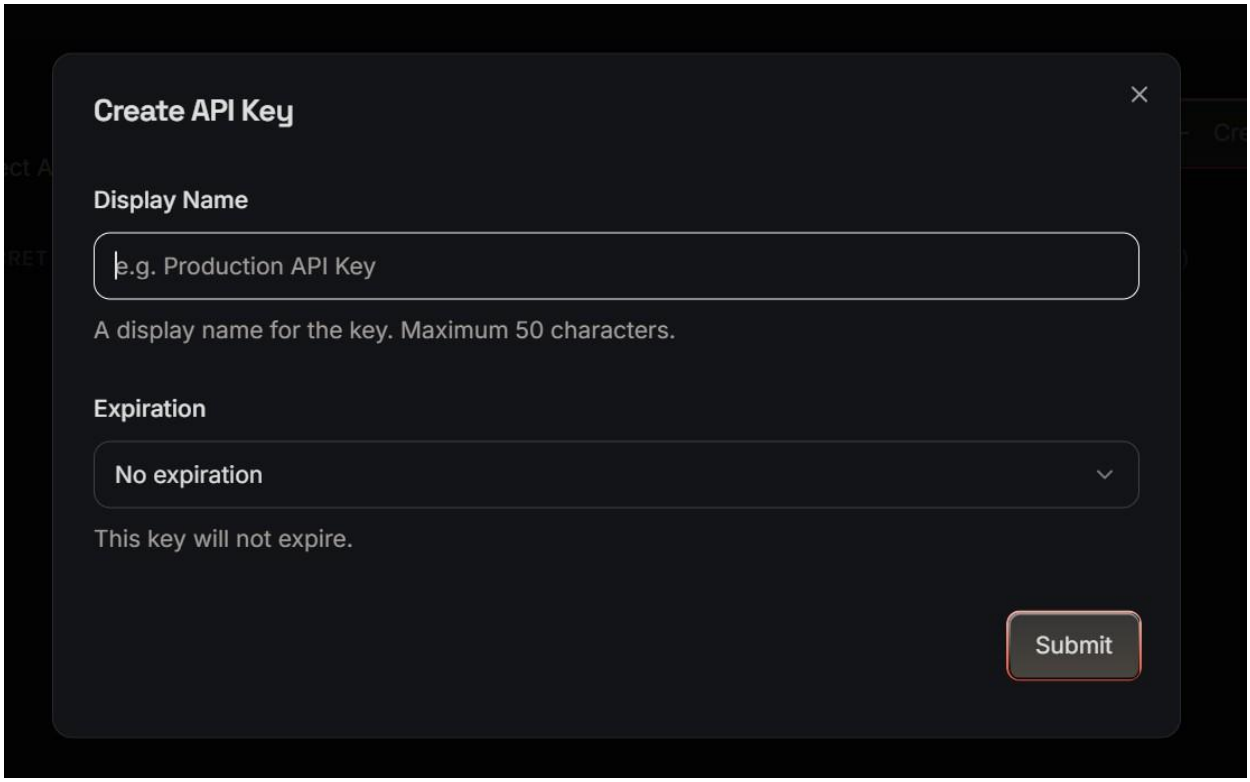
• **Step 2 — Navigate to API Keys**: After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the API Keys page.



• **Step 3 — Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (e.g., captionai-key) so it is easy to identify later.



• **Step 4 — API Key:** Now give a "Display Name" and "Expiration" is optional, and Click on "Submit". The you will get your API key, Copy it immediately and store it in a safe place you will not be able to view it again after closing the dialog.



• **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:
GROQ_API_KEY=your_copied_api_key_here
• **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

• **Important** — Never Commit Your API Key: Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select a best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the CaptionAI application, focusing on the types of content it will generate (captions, hashtags, and calls-to-action across Instagram and LinkedIn).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to social media copywriting tasks.
- **Select the Optimal Model:** Choose **llama-3.3-70b-versatile** for its balance of creative output quality and generation speed, set with a temperature of 0.8 and max_tokens of 2048 for rich, varied captions.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application entering product/campaign information, selecting a platform (Instagram or LinkedIn), and choosing a tone (Professional, Casual, or Promotional).
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using TONE_DESCRIPTIONS and PLATFORM_TIPS dictionaries, calls the Groq API, parses the JSON response via extract_json(), and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv

D:\projects>"d:\projects\venv\Scripts\activate"

(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects> pip install groq==0.11.0
```

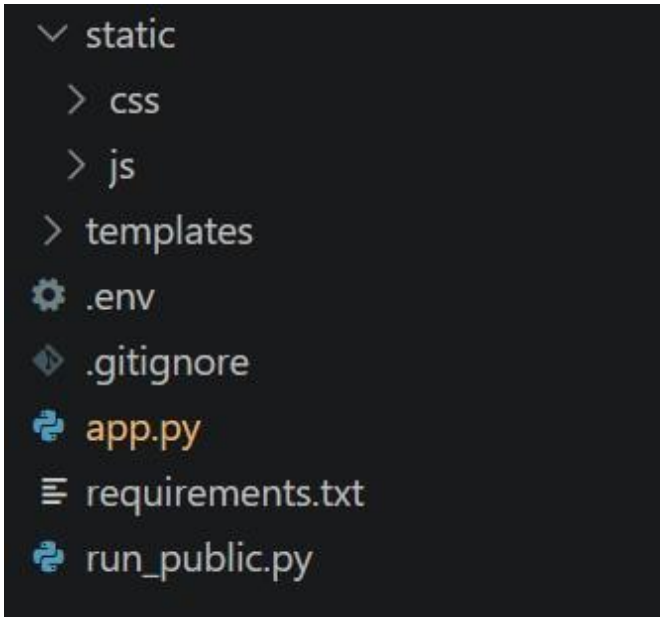
- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:

  GROQ_API_KEY=your_groq_api_key_here

- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features Captions, Hashtags, and CTAs as a unified AI response driven by a single structured prompt.

- **Caption Generator:** Generates three unique, platform-appropriate captions. For Instagram/Casual tone, includes relevant emojis and conversational language. For LinkedIn/Professional tone, produces insight-led, value-driven copy.
- **Hashtag Suggester:** Produces ten hashtags per request a curated mix of high-reach popular tags and targeted niche tags relevant to the product or campaign.
- **CTA Generator:** Returns three short (under 10 words), punchy, action-oriented call-to-action phrases calibrated to the selected platform and tone.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the home page and content generation endpoint.

Process User Input:

- Create a form in index.html for users to submit their product/campaign information, platform, and tone.
- Use Flask's request.get_json() to retrieve JSON data submitted from the frontend via AJAX.
- Validate that product_info is non-empty and within the 2000-character limit before processing.

Integrate API Calls:

- Within the /generate route handler, call the build_prompt() function to construct a structured prompt combining tone descriptions and platform-specific tips.
- Submit the prompt to the Groq API using client.chat.completions.create() with the llama-3.3-70b-versatile model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for CaptionAI's two endpoints, each serving a distinct purpose:
  - **/** — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

  - **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```
@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()
    product_info = data.get("product_info", "").strip()
    platform = data.get("platform", "instagram").lower()
    tone = data.get("tone", "professional").lower()
    if not product_info:
        return jsonify({"error": "Product information is required."}), 400
    if len(product_info) > 2000:
        return jsonify({"error": "Input too long. Please keep it under 2000 characters."}), 400
    try:
        prompt = build_prompt(product_info, platform, tone)

        chat_completion = client.chat.completions.create(
            messages=[
                {
                    "role": "system",
                    "content": "You are a social media content expert. Always respond with valid JSON
only, no markdown formatting."
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            model="llama-3.3-70b-versatile",
            temperature=0.8,
            max_tokens=2048,
```

```
        )
        response_text = chat_completion.choices[0].message.content
        result = extract_json(response_text)
        # Validate structure
        if not all(k in result for k in ["captions", "hashtags", "ctas"]):
            raise ValueError("Invalid response structure from AI")
        return jsonify({
            "success": True,
            "captions": result["captions"],
            "hashtags": result["hashtags"],
            "ctas": result["ctas"],
            "platform": platform,
            "tone": tone
        })
    except ValueError as e:
        import traceback
        traceback.print_exc()
        return jsonify({"error": f"Failed to parse AI response: {str(e)}"}), 500
    except Exception as e:
        import traceback
        traceback.print_exc()
        return jsonify({"error": f"Generation failed: {str(e)}"}), 500
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```
TONE_DESCRIPTIONS = {
    "professional": "formal, authoritative, and business-oriented. Use clear, concise language that conveys expertise and credibility.",
    "casual": "friendly, conversational, and relatable. Use informal language, emojis, and a warm tone that feels personal.",
    "promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers to drive engagement."
}
```

- **PLATFORM_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).

```
PLATFORM_TIPS = {
    "instagram": "optimized for Instagram — visually descriptive, emotionally engaging, with strong visual storytelling. Max 2200 characters.",
    "linkedin": "optimized for LinkedIn — professional insights, value-driven, thought leadership style. Max 3000 characters."
}
```

- **build_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

```
def build_prompt(product_info: str, platform: str, tone: str) -> str:
    tone_desc = TONE_DESCRIPTIONS.get(tone, "engaging")
    platform_tip = PLATFORM_TIPS.get(platform, "social media")

    return f"""You are an expert social media content strategist and copywriter.
```

Integrate the Groq AI Responses in Each Function:

- Call client.chat.completions.create() with a system message enforcing JSON-only responses and a user message containing the assembled prompt.
- **extract_json()**: a robust parser that first attempts direct json.loads(), then strips markdown code fences (```json blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    # Try direct parse first
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    # Try extracting from markdown code block
    match = re.search(r'```(?:json)?\s*([\s\S]*?)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r'\{[\s\S]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains all three required keys — captions, hashtags, and ctas — before returning it to the frontend.
- **Home route:** / → renders index.html/generate (POST) → accepts JSON, returns {success,captions, hashtags, ctas, platform, tone}

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the CaptionAI brand logo and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for platform indicators and result section headings, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark glassmorphism aesthetic with animated background blobs (blob-1, blob-2, blob-3) for visual depth.
- Use flexbox and grid layouts to organize the results into a two-column structure (captions on the left, hashtags and CTAs on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Caption Cards**: individual cards per caption, each with a one-click copy-to-clipboard button that provides visual feedback (checkmark + green highlight for 2 seconds).
- **Hashtag Chips**: rendered as pill-shaped chip elements inside a glass card, with automatic # prefix validation.
- **CTA List**: displayed as a clean unordered list inside a separate glass card with a bullhorn icon header.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

Render Results Dynamically:

- Implement the renderResults(data) function in main.js, which dynamically creates and injects DOM elements for captions, hashtags, and CTAs into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollIntoView().

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

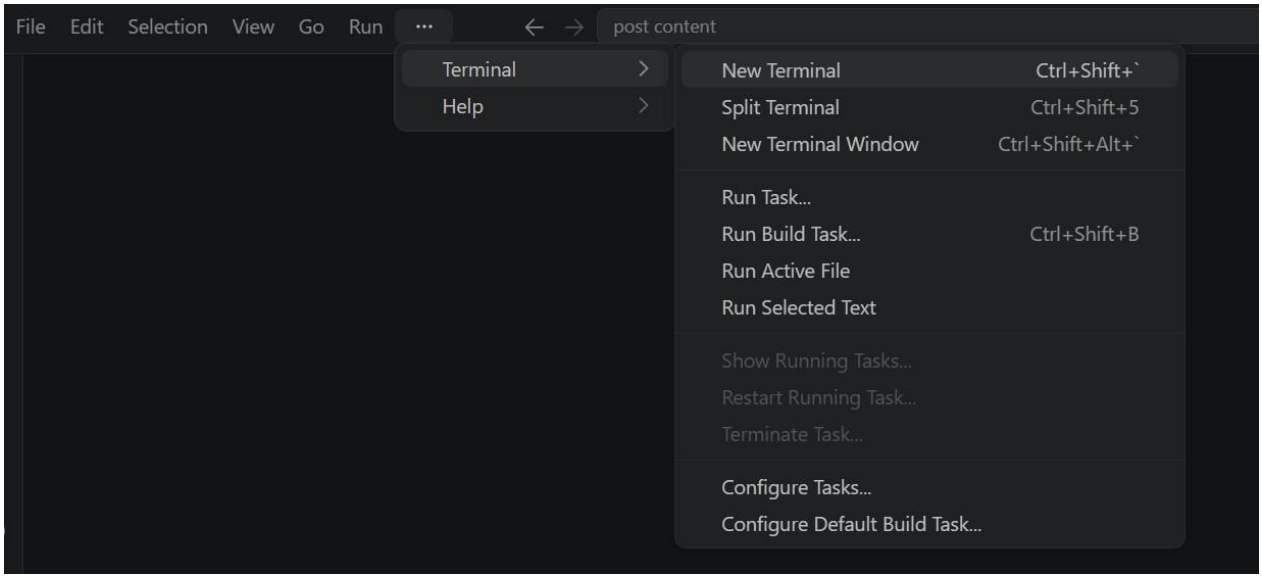
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

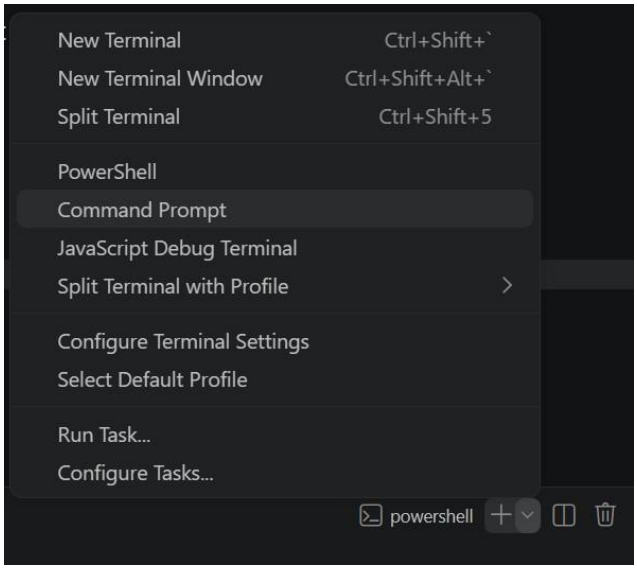
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:

  GROQ_API_KEY=your_groq_api_key_here

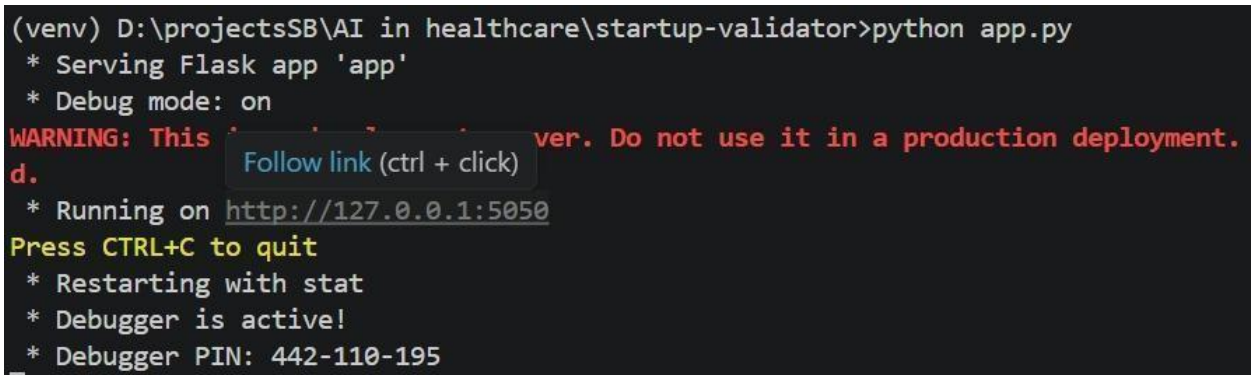
Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.



- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

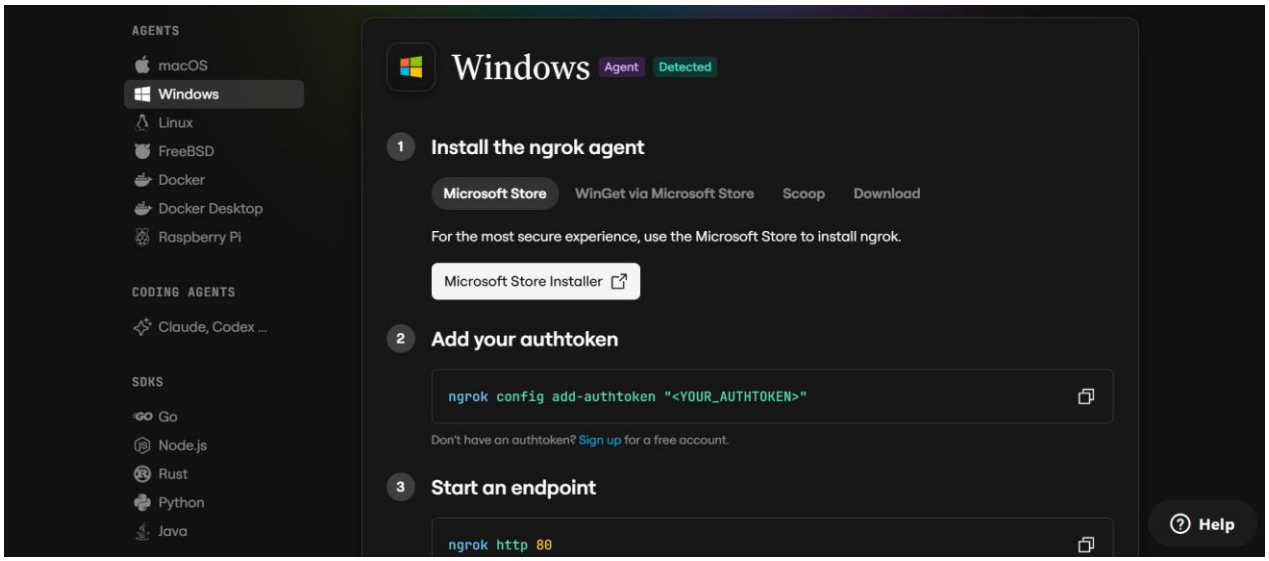
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from https://ngrok.com/download.
- Click on "Microsoft Store Installer".



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

  ==========

  YOUR PUBLIC WEBSITE URL IS LIVE!

  URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app

  ==========

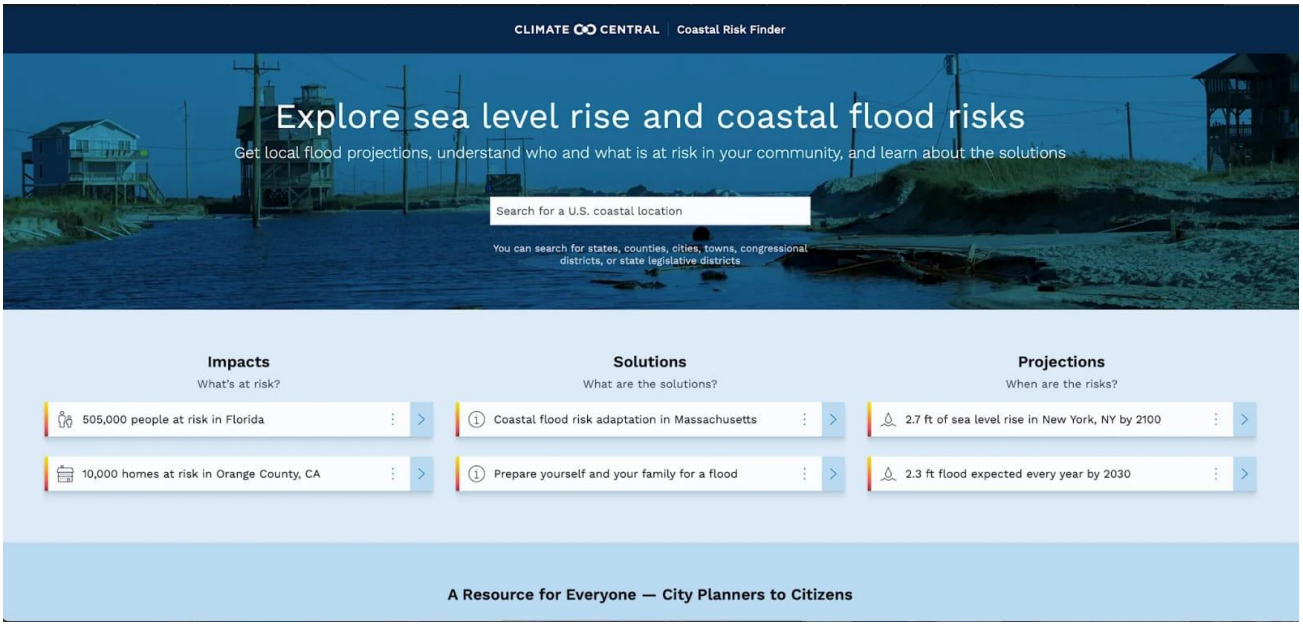
- Share this URL with anyone who can access your CaptionAI app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.

Exploring the Website's Web Pages:

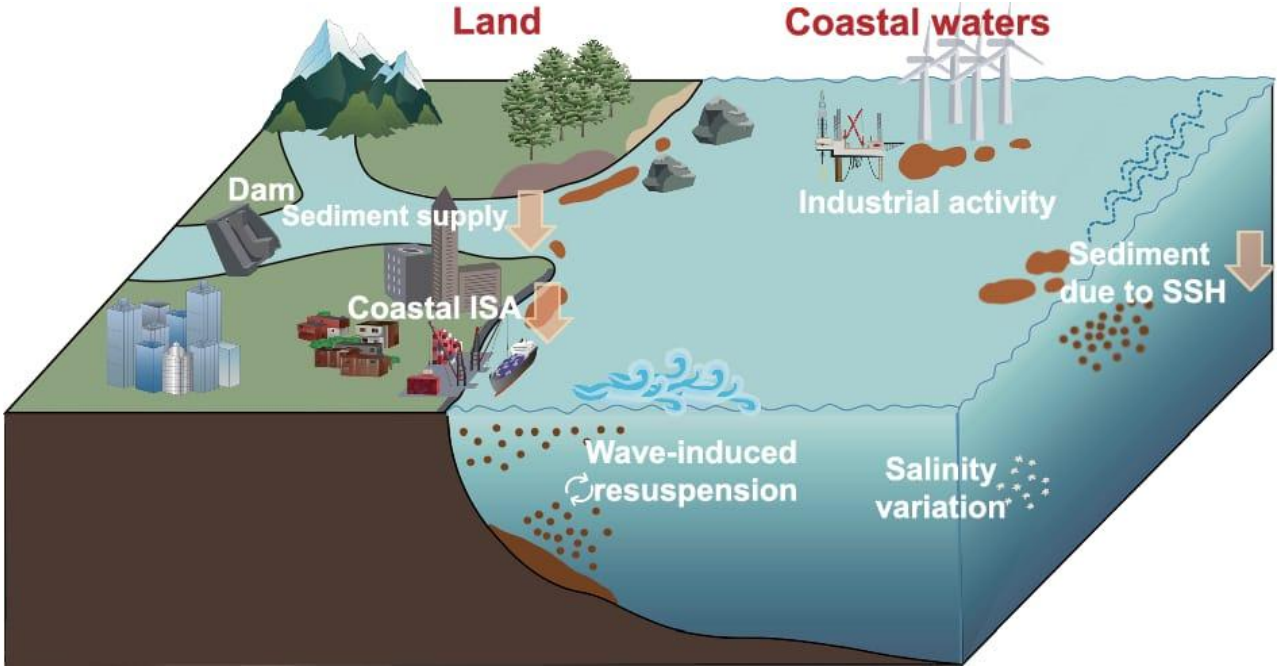
Home / Generator Page:



Description:
data preprocessing was optimized by removing missing values and duplicate records. The machine learning model parameters were tuned to improve prediction accuracy. Unnecessary computations were reduced, resulting in faster execution and efficient resource utilization

Input Section:

Results Section:



Description:

The Flood Prediction System successfully analyzes environmental parameters such as rainfall, river water level, humidity, temperature, and soil moisture to predict the likelihood of flooding. After preprocessing the dataset and training the machine learning model, the system generates accuration

data preprocessing was optimized by removing missing values and duplicate records. The machine learning model parameters were tuned to improve prediction accuracy. Unnecessary computations were reduced, resulting in faster execution and efficient resource utilizatio

Conclusion:

The Flood Prediction System was successfully developed and tested using machine learning techniques. The system effectively analyzes historical flood data and environmental factors to predict flood risk with high accuracy. Data preprocessing, model training, and performance evaluation improved the reliability of predictions. This project can support early flood warning systems, help disaster management authorities make timely decisions, and reduce the impact of floods on human life and property. In the future, the system can be enhanced by integrating real-time weather data, IoT sensors, and GIS mapping for more accurate and efficient flood prediction